

Doc-Intelligence-PID

PowerPoint summary of the ultra-modern architecture and processing flow

AI Document Intelligence for Personal Insurance Documents

Frontend: ReactJS • Backend: FastAPI • Retrieval: Agentic AI + FAISS • vLLM with Vision (for OCR): Qwen2.5

Output: Extracted text and PID data Served by FastAPI and rendered by React UI

Customized and Restricted Backend Filters are used as Guard Rail

Target Audience

Personal Insurance Service Executives

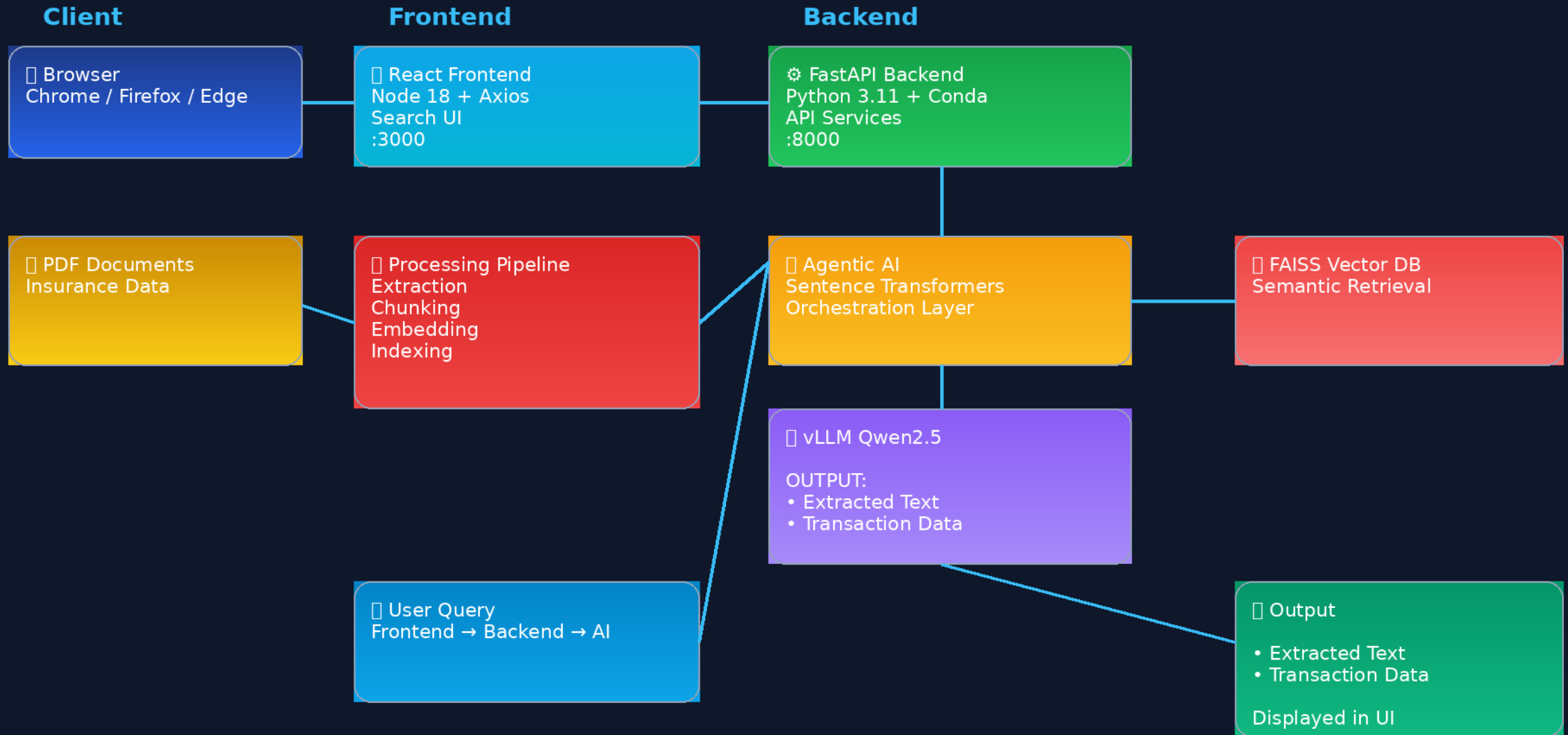
Personal Insurance SPoC

Serving existing PID data

(Not PID Sales)

Doc-Intelligence-PID

AI Document Intelligence Architecture



Core Components

What each layer is responsible for

Frontend

ReactJS on Node 18
Axios for API communication
UI for search, upload, and result display

Backend

FastAPI on Python 3.11
Conda environment
Request routing and result serving

Agentic AI

Sentence Transformers mini LLM
Request/response orchestration
Context building for retrieval

FAISS

Vector index
Top-K semantic retrieval

vLLM / Qwen2.5

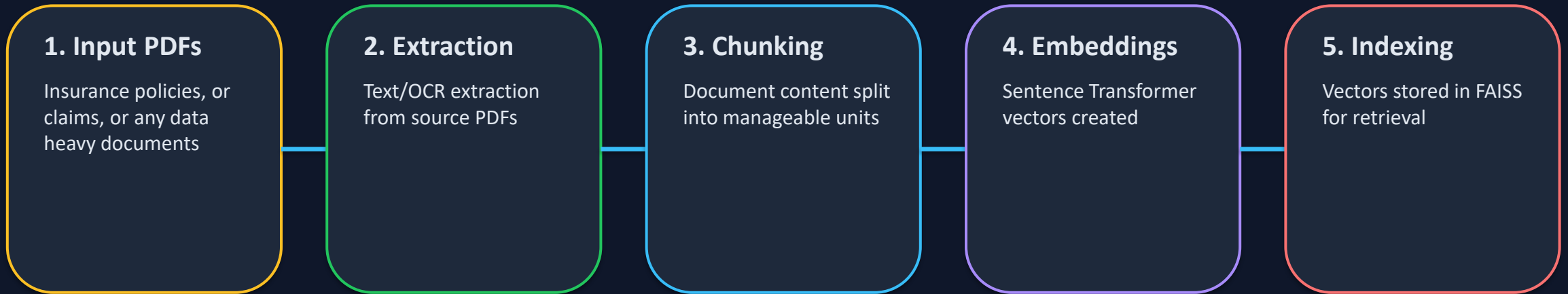
Processes retrieved context
Extracts text and transaction data
Powers final responses

Document Pipeline

PDF ingestion
Extraction, chunking, embeddings, indexing
Feeds searchable content into FAISS

Ingestion Pipeline

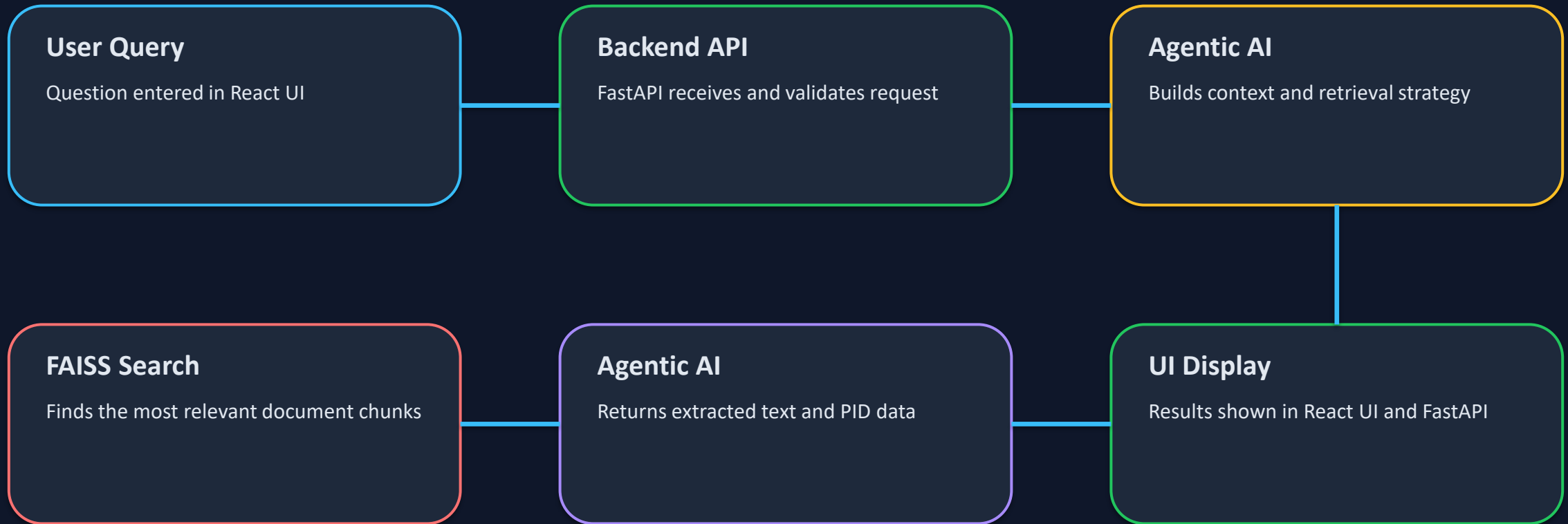
How PDF documents become searchable assets



This pipeline prepares the local knowledge base for semantic search.
The output of this stage is a searchable FAISS index used at query time.

Runtime Query Flow

What happens when a user asks a question



Output Experience

What the user sees and what the system returns

Served Data

Extracted text from PDF content
PID data identified by the model
Results are tailored for local document intelligence workflows

Rendering Display Content

ReactJS frontend for end-user interaction
FastAPI-backed service layer for delivery and API integration
Consistent result presentation across the application flow

This design keeps the interaction simple: query in, extracted text and transaction-focused results out.

Tech Stack Summary

project stack organized by layers

Environment

Ubuntu / Windows
Conda environment
Localhost deployment

Frontend

NodeJS 18
ReactJS
Axios
Browser support: Chrome,
Firefox, Edge
React rendered at
localhost:3000

Backend

Python 3.11
FastAPI
Served at localhost:8000

AI / Search

Sentence Transformers for AgenticAI
(Mini LM L5 v2)
FAISS-DB
Qwen2.5 via vLLM

Presentation Talking Points

Business Use Case: Dynamically searching and extracting PID information from PDF documents

Ingestion pipeline builds the FAISS index, and a Runtime pipeline answers user queries

Agentic AI act as the orchestration layer between FastAPI, FAISS

User-facing output: extracted text and PID data responded in FastAPI and rendered in React UI

Deployment model: local, privacy-friendly, modular for future scaling

Sematic + NLP content restricted filters are used as GUARD Rail